

SQL Joins from Scratch: What, Why and Use Cases

1. What is a JOIN?

- A JOIN is used to combine data from two or more tables.
- It works based on a common column between tables.
- In real projects, data is usually stored in multiple tables.
- JOIN helps us bring related data together.

Simple meaning:

JOIN = connecting related tables to get meaningful output

Example:

customers.customer_id = orders.customer_id

Here, `customer_id` is the common column.

2. Why Do We Need Joins?

In real projects, we do not store everything in one big table.

Example:

- Customer details are stored in `customers`
- Order details are stored in `orders`
- Product details are stored in `products`
- Payment details are stored in `payments`

If business asks:

Show customer name, city, order amount and order date

Then we need data from both:

customers + orders

So we use JOIN.

3. Why Not Store Everything in One Table?

If we store everything in one table, data will repeat again and again.

Example:

order_id	customer_name	city	order_amount
101	Anurag	Bengaluru	50000
102	Anurag	Bengaluru	1000

Problem:

- Customer name is repeated
- City is repeated
- More storage is used
- Update becomes difficult
- Data can become inconsistent

Better design:

customers table → customer details

orders table → order details

Then we connect both using JOIN.

4. Relation Between Keys and Joins

Keys and joins are directly connected.

- Primary Key uniquely identifies a row in one table.

- Foreign Key connects one table with another table.
- JOIN uses this relationship to combine data.

Example:

`customers.customer_id = orders.customer_id`

Here:

- `customers.customer_id` can be Primary Key
- `orders.customer_id` can be Foreign Key

Important line:

Keys define the relationship.

Joins use that relationship.

5. Basic JOIN Syntax

```
SELECT
    table1.column_name,
    table2.column_name
FROM table1
JOIN table2
    ON table1.common_column = table2.common_column;
```

Example:

```
SELECT
    c.customer_name,
    c.city,
    o.order_id,
    o.order_amount
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id;
```

6. Simple Example

customers table

customer_id	customer_name	city
1	Anurag	Bengaluru
2	Stuti	Pune
3	Rahul	Delhi
4	Priya	Mumbai

orders table

order_id	customer_id	order_amount
101	1	50000
102	1	1000
103	2	2500
104	5	1500

Join query:

```
SELECT
  c.customer_id,
  c.customer_name,
  c.city,
  o.order_id,
  o.order_amount
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id;
```

Output:

customer_id	customer_name	city	order_id	order_amount
1	Anurag	Bengaluru	101	50000

1	Anurag	Bengaluru	102	1000
2	Stuti	Pune	103	2500

7. Important Observations

- Anurag appears two times because he has two orders.
 - This is not always duplicate data.
 - It is a one-to-many relationship.
 - Rahul and Priya are not shown because they have no matching orders.
 - Order 104 is not shown because customer ID 5 does not exist in customers.
-

8. Practical Use Cases of Joins

1. Reporting

Used to prepare business reports.

Example:

Customer name + Order amount + Payment status

2. Data Enrichment

Raw tables usually contain IDs.

Example:

customer_id, product_id, payment_id

Business users need names and details.

So we join with master tables to add meaning.

3. Data Validation

Joins help us find:

- Missing customers
 - Invalid orders
 - Unmatched records
 - Bad data quality
-

4. Data Migration Testing

During migration, joins are used to compare:

- Source table
- Target table

We can check:

- Missing records
 - Extra records
 - Mismatched values
-

5. Data Warehouse and Analytics

In data warehouses, joins are used between:

fact tables + dimension tables

Example:

fact_orders + dim_customer + dim_product

This is very common in dashboards and analytics.

9. Types of Joins We Will Learn

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN

- CROSS JOIN
- SELF JOIN
- NON-EQUI JOIN
- SEMI JOIN
- ANTI JOIN
- NATURAL JOIN

Some joins are used frequently.

Some joins are rarely used.

Some joins should be avoided in production unless clearly needed.

10. Common Mistakes

Mistake 1: Forgetting the ON condition

Wrong:

```
SELECT *  
FROM customers c  
JOIN orders o;
```

Correct:

```
SELECT *  
FROM customers c  
JOIN orders o  
  ON c.customer_id = o.customer_id;
```

Mistake 2: Joining on the wrong column

Wrong:

```
ON c.customer_name = o.customer_id
```

Correct:

```
ON c.customer_id = o.customer_id
```

Mistake 3: Thinking repeated rows are always duplicates

If one customer has multiple orders, the customer will appear multiple times.

This is expected.

Do not use **DISTINCT** blindly.

First understand the business requirement.

11. Revision Summary

- JOIN combines data from multiple tables.
 - It works using a common column.
 - Joins are connected with Primary Key and Foreign Key.
 - Joins are used in reporting, ETL, validation, migration and analytics.
 - Repeated rows after join are not always wrong.
 - The **ON** condition is very important.
 - Before writing a join, understand the business question.
-

Interview Questions and Answers

Q1. What is a JOIN in SQL?

A JOIN is used to combine data from two or more tables based on a related column between them.

Q2. Why do we need joins?

We need joins because real-world data is stored in multiple related tables. Joins help us bring that data together for reporting, analysis and ETL work.

Q3. What is the relation between keys and joins?

Keys define the relationship between tables. Joins use that relationship to combine data.

Example:

`customers.customer_id = orders.customer_id`

Q4. Why should we not store everything in one big table?

Because it creates duplicate data, update issues, storage problems and data inconsistency.

Q5. Can we join more than two tables?

Yes. In real projects, we often join multiple tables.

Example:

`customers + orders + products + payments`

Q6. Why do repeated rows come after a join?

Repeated rows can come because of one-to-many or many-to-many relationships.

Example:

One customer can place multiple orders.

So the customer will appear once for every matching order.

Q7. Is repeated data after join always wrong?

No. It depends on the business requirement.

If we need order-level data, repeated customer names are valid.

If we need customer-level data, we may need aggregation.

Q8. What happens if we forget the ON condition?

The query may fail or create wrong combinations depending on the database.

This can lead to incorrect output and a very large number of rows.

Q9. What is the most important part of a JOIN query?

The **ON** condition.

It tells SQL how both tables are related.

Q10. Where are joins used in data engineering?

Joins are used in:

- ETL pipelines
 - Data enrichment
 - Reporting
 - Data validation
 - Data migration testing
 - Data warehouse transformations
 - Dashboard preparation
-

Q11. What should we check before writing a join?

Check:

- What is the business requirement?
- Which tables are needed?
- What is the common column?
- Is the join key unique?
- Do we need matching records only or unmatched records also?

Q12. What is one common beginner mistake in joins?

Using **DISTINCT** directly when repeated rows appear.

Repeated rows may be valid because of one-to-many relationships.

First understand the grain of the data.

Small Practice Questions

Practice 1

Show customer name, city, order ID and order amount.

```
SELECT
  c.customer_name,
  c.city,
  o.order_id,
  o.order_amount
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id;
```

Practice 2

Which column connects customers and orders?

Answer:

customer_id

Practice 3

Why is Anurag appearing two times?

Answer:

Because Anurag has two orders.

Practice 4

Why are Rahul and Priya not visible in the output?

Answer:

Because they do not have matching records in the orders table.

Practice 5

Why is order ID 104 not visible?

Answer:

Because order ID 104 has customer ID 5, but customer ID 5 does not exist in the customers table.